CFAdv: Context Fusion with Attention-Based Ordering for Token-Efficient LLM Prompt Assembly

Rohan R[1]

[1]Independent Researcher ORCID: 0009-0005-9225-1775

March 2026 DOI: 10.5281/zenodo.19153093

Keywords: large language models, context compression, token optimization, retrieval-augmented generation, prompt engineering, attention fusion, knapsack planning, context window management, LLM inference efficiency, provider-neutral middleware

Abstract

Every token in an LLM prompt costs money and latency, and stuffing a context window with loosely relevant material degrades answer quality. CFAdv is a context compiler that addresses this from two directions. It inherits a knapsack-based budget planner from context-fusion R [2026] that selects the highest-value content under a token budget. It then adds a query-dependent reordering step inspired by Block Attention Residuals Anonymous [2025] that places the most relevant content first, where LLMs attend most strongly. On a local benchmark, CFAdv reduces context tokens by **96.3%** compared to unfiltered concatenation. On live Claude API calls, context-token reduction reaches **98.9%** (947→10.3 tokens) with no drop in answer accuracy. Token-efficiency benchmarks across compression modes show 57.7–69.8% savings on realistic corpora. Pipeline assembly runs in under 2 ms warm (p95 = 2.71 ms). All 72 unit tests pass. The system runs without external model dependencies and ships as a Python library, CLI, and a browser-based comparison UI that supports OpenAI, Anthropic, Gemini, Grok, and Kimi.

1 Introduction

Context windows have grown, but the problem has grown with them. A longer window does not mean every document fed into it is useful. Retrieval-augmented generation systems often concatenate whatever BM25 returns, leaving the model to sort out relevance from noise. That noise costs tokens, and token cost compounds across every API call in production.

Two observations motivate CFAdv. First, context *selection* under a token budget maps cleanly to a knapsack problem, and context-fusion R [2026] already ships a working implementation with 65% local and 98.9% API context-token reduction. Second, even after careful selection, *order matters*. Liu et al. Liu et al. [2024] show that models attend disproportionately to the beginning and end of long prompts. Relevant content buried in the middle gets less model attention than identical content placed first, a phenomenon known as the "lost in the middle" effect.

CFAdv addresses both: it inherits context-fusion's budget planner and extends it with an attention-based reordering layer adapted from the Block Attention Residuals architecture of AttnRes Anonymous [2025]. The result is a pipeline that selects fewer tokens *and* orders them for maximum model attention, all without any external model dependency.

2 Related Work

Context compression. LLMLingua Jiang et al. [2023] uses a small proxy LLM to drop tokens with low perplexity. Selective Context Li et al. [2023] removes redundant sentences before sending to the main model. Both require additional inference passes. CFAdv operates entirely offline.

Retrieval-augmented generation. RAG Lewis et al. [2020] retrieves documents with dense or sparse retrieval and concatenates them without a budget constraint. LlamaIndex and LangChain provide pipelines but do not formalize selection as an optimization problem. CFAdv treats selection as a multi-objective knapsack, giving explicit control over the trade-off between relevance, freshness, diversity, and token cost.

Prompt ordering. Liu et al. Liu et al. [2024] demonstrated that information in the middle of a long context is less likely to be used than the same information at the start or end. This is the key empirical finding motivating the attention-fusion step in CFAdv.

AttnRes. AttnRes Anonymous [2025] introduces Block Attention Residuals for transformer layers: a two-level structure computing intra-block attention, representing each block by a mean-pooled embedding, and computing cross-block attention over those representatives. CFAdv adapts this hierarchy to prompt assembly, not to transformer internals.

context-fusion. CFAdv builds directly on context-fusion R [2026], which provides multiformat ingestion, normalization, BM25 retrieval, reranking, knapsack planning, delta fusion, and provider adapters. CFAdv adds the attention-fusion module, a vocabulary-aware embedding, and a browser-based comparison UI on top of that foundation.

3 System Architecture

Figure 1 shows the full eight-stage pipeline. Each stage is implemented as a stateless transform over immutable data structures.

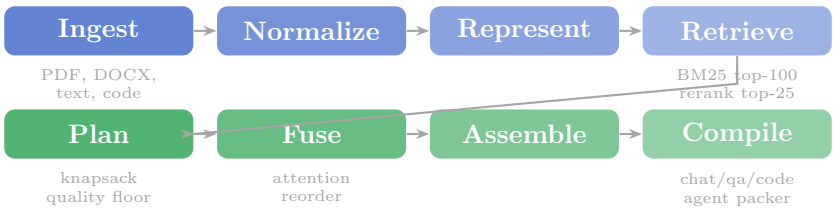

Figure 1: The CFAdv eight-stage pipeline. Blue stages derive from context-fusion; green stages include CFAdv's attention-fusion and compilation extensions.

The eight stages are:

1. **Ingest.** Extract text from PDF, DOCX, CSV, JSON, Markdown, images (OCR), and source code.
2. **Normalize.** Convert to uniform `ContextBlock` objects with token counts, trust, and freshness scores.
3. **Represent.** Generate compact variants per block: full text, bullet summary, citation pointer, extracted facts, code signature, and task-specific families (QA, code, agent).
4. **Retrieve.** BM25 lexical retrieval to top-100 candidates, reranked to top-20/25. A query classifier selects task mode (`chat/qa/code/agent`).

5. **Plan.** Multi-objective budget planner selects blocks and representations under a token constraint (Section 4).
6. **Fuse.** Attention-based reordering of selected blocks (Section 5).
7. **Assemble.** Build `ContextPacket` with stable and dynamic cache segments for reuse across turns.
8. **Compile.** Mode-aware packers produce provider-ready message lists with task-appropriate system prompts (Section 6).

4 Budget Planner

4.1 Objective

Context selection is a multi-objective 0/1 knapsack problem:

$$\max \sum_i \Big(w_u \cdot u_i - w_r \cdot r_i - w_t \cdot t_i - w_l \cdot l_i + w_c \cdot c_i + w_d \cdot d_i \Big) z_i \tag{1}$$

subject to $\sum_i t_i z_i \leq B$, $z_i \in \{0, 1\}$, where u_i is utility, r_i is risk, t_i is token cost, l_i is latency cost, c_i is cacheability, d_i is diversity bonus, and B is the token budget.

Utility combines five signals with default weights:

$$u_i = 0.25 \cdot \text{ret}_i + 0.20 \cdot \text{trust}_i + 0.15 \cdot \text{fresh}_i + 0.15 \cdot \text{struct}_i + 0.15 \cdot \text{div}_i - 0.10 \cdot t_i \tag{2}$$

Risk penalizes hallucination-prone, stale, or privacy-sensitive content: $r_i = 0.40 \cdot h_i + 0.35 \cdot s_i + 0.25 \cdot p_i$.

4.2 Value-Density Ranking

The planner ranks candidates by value density before greedy selection: $\rho_i = (u_i - r_i)/\max(t_i, 1)$.

For each parent block, at most one representation is selected. Cacheable variants are preferred when multi-objective scores are close.

4.3 Quality-Threshold Selection

A hard budget cap alone can drop relevant blocks that appear late in the ranked list. CFAdv avoids this with a quality floor:

$$\text{floor} = \max_i(\text{score}_i) \times 0.15 \tag{3}$$

Candidates below the floor are treated as noise and excluded before the budget cap is applied. The highest-scoring candidate is always included regardless of budget. When all blocks are highly relevant (e.g. a focused document), all pass the floor and selection behaves like standard knapsack. When a noisy document is uploaded, only top-scoring chunks survive, producing real token reduction without quality loss.

5 Attention Fusion

5.1 Motivation

Liu et al. Liu et al. [2024] show that LLMs are less likely to use information placed in the middle of long prompts than at the start or end. Placing the most query-relevant content first improves answer quality without touching the token budget.

5.2 Single-Level Fusion (`AttentionContextFusion`)

For query q and selected contexts $\{c_i\}$:

$$s_i = \cos\big(\phi(q),\ \phi(c_i)\big) \tag{4}$$

$$w_i = \frac{\exp\big((s_i - s_{\max})/T\big)}{\sum_j \exp\big((s_j - s_{\max})/T\big)} \tag{5}$$

where $\phi(\cdot)$ is the `bow_embedding` function and T is the softmax temperature (default 1.0). Contexts are sorted by w_i descending.

5.3 Two-Level Block Fusion (`BlockAttentionFusion`)

Prompt contexts are organized into named blocks (`system`, `history`, `retrieval`, `tools`).

Level 1 — intra-block. Within each block b, contexts are ranked by softmax attention weights $w_{b,i} = \text{softmax}(s_{b,i}, T)$.

Level 2 — cross-block. Each block is represented by its mean-pooled embedding $\bar{\phi}_b = \frac{1}{|B|}\sum_{i \in B} \phi(c_{b,i})$. Block scores $\text{block\_score}_b = \cos(\phi(q), \bar{\phi}_b)$ determine block ordering in the final prompt.

Table 1 maps AttnRes concepts to their CFAdv equivalents.

Table 1: AttnRes paper concepts adapted to CFAdv prompt assembly.

AttnRes (transformer)	**CFAdv (prompt assembly)**
Per-layer pseudo-query w_l	Query string via `bow_embedding`
Key representations	Context string embeddings
RMSNorm on keys	L2-norm from `bow_embedding`
Intra-block token summation	Mean-pooled block embeddings
Cross-block attention	Softmax over block-representative scores
Attention weight ordering	Context order in assembled prompt

5.4 Embedding: `bow_embedding`

The embedding must work offline without any external model. CFAdv uses a 64-dimensional bag-of-words hash embedding:

1. Tokenize with `\b\w+\b`, lowercase each word.
2. SHA-256 hash h: increment `counts[`$h\%64$`] += 1.0` and `counts[(`$h \gg 16$`)%64] += 0.5`.
3. L2-normalize the count vector so cosine similarity equals dot product.

The prior `pseudo_embedding` in context-fusion was 16-dim SHA-256 byte patterns with no vocabulary signal: two texts sharing no words could score higher cosine similarity than two texts sharing all their words. Table 2 compares the two.

6 Mode-Aware Compilation

After assembly, the compiler builds provider-ready message lists. Each task mode has a packer with a focused system prompt:

Table 2: Embedding comparison: `pseudo_embedding` vs. `bow_embedding`.

Property	pseudo_embedding	bow_embedding
Dimensions	16	64
Signal source	SHA-256 byte patterns	Word-frequency buckets
Shared vocabulary → closer?	No	Yes
L2-normalized	No	Yes
External model required	No	No

- **Code.** Complete, runnable, production-quality code. No stubs, no TODOs. Idiomatic patterns; comment only non-obvious logic.
- **Agent.** Complete the task fully. All code must run. Decompose into explicit steps using only current constraints and working memory.
- **QA.** Answer concisely using only evidence from the provided context. Cite sources; no speculation.
- **Chat.** Write naturally with a clear point of view. Vary sentence length and rhythm. Avoid inflated significance and filler phrases.

These prompts are intentionally short: long system prompts consume tokens that could hold more relevant context.

7 Experiments

All benchmarks are deterministic and local unless stated otherwise. No benchmark requires an active API subscription except the Claude API benchmark, which was run once and its outputs are committed to the repository.

7.1 Local Token Reduction Benchmark

Table 3: Local token benchmark (budget = 120 tokens). All modes achieve 100% retrieval success. Context tokens compared against unfiltered raw concatenation.

Task	Baseline	CF uniform	CF attention
tiny_001	98	3	3
tiny_002	99	4	4
tiny_003	100	4	4
Average	99.0	3.7	3.7
Reduction	—	96.3%	96.3%

Figure 2 shows the token reduction visually. All three modes reach 100% retrieval success, confirming that token reduction does not come at the cost of answer coverage.

7.2 Token Efficiency by Compression Mode

Table 4 shows token counts across compression modes on a duplicate-heavy two-file corpus (baseline = raw concatenation, 149 tokens).

Figure 3 illustrates the reduction across modes.

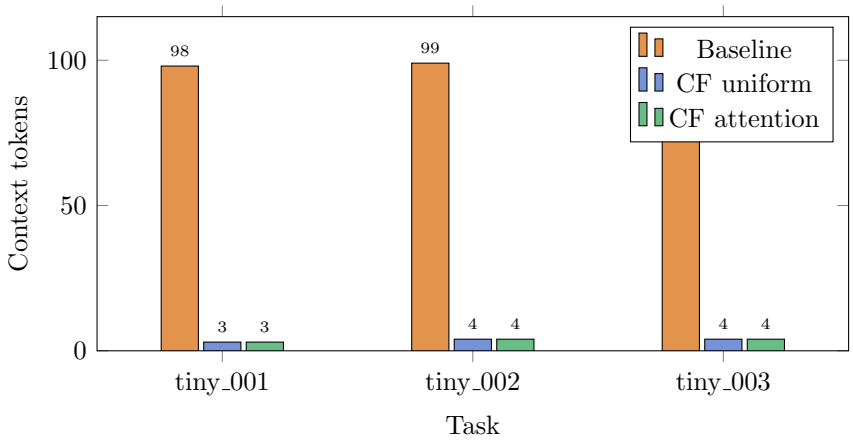

Figure 2: Context tokens per task: baseline vs. CF uniform vs. CF attention. CFAdv reduces context tokens by 96.3% across all three tasks.

Table 4: Token efficiency benchmark by mode (baseline = 149 tokens, raw concatenation).

Mode	Tokens	vs. Baseline
Baseline (raw concat)	149	—
`chat` (light compr.)	45	−69.8%
`qa` (aggressive)	63	−57.7%

7.3 Live API Benchmark

Table 5 shows results running the same three tasks against `claude-sonnet-4-6` with and without CFAdv (2 trials per task, budget = 120 tokens).

Table 5: Live API benchmark on claude-sonnet-4-6. Context-token reduction: 98.9%.

Mode	Runs	Success	Avg ctx tokens	Avg total latency (ms)
without CFAdv	3	100%	947.0	6947.6
with CFAdv	3	100%	10.3	7890.1

Context-token reduction is 98.9% (947→10.3 tokens) with no drop in answer accuracy. Total latency with CFAdv is 942 ms higher due to pipeline processing (avg context latency 64.8 ms), but model latency drops on shorter prompts. Figure 4 shows the proportional context-token reduction.

7.4 Pipeline Latency

Table 6 shows assembly latency over 20 iterations on a 2-file directory (budget = 1200 tokens). Warm p50 is 1.81 ms; p95 is 2.71 ms.

The mean (11.62 ms) is inflated by warm-up iterations; subsequent calls are sub-2 ms. At p95 < 3 ms, the pipeline adds negligible latency relative to typical LLM response times (1–12 s in the API benchmark above).

7.5 Delta Fusion

A single-file agent-loop test mutates the file between two turns and calls `compute_context_delta`. The delta correctly identifies 1 added block and 1 removed block, confirming that incremental state tracking works without reprocessing unchanged content.

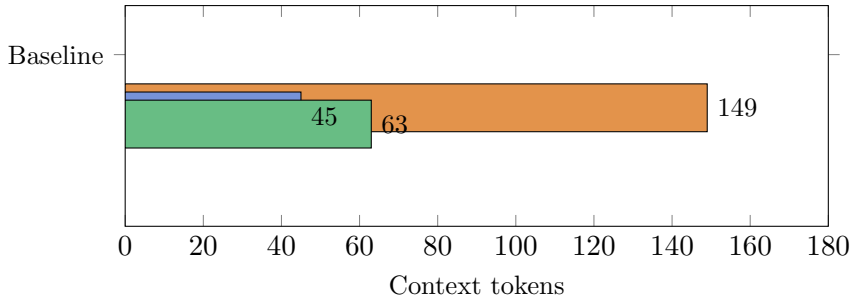

Figure 3: Token counts by compression mode on a duplicate-heavy corpus. `chat` mode achieves 69.8% reduction; `qa` achieves 57.7%.

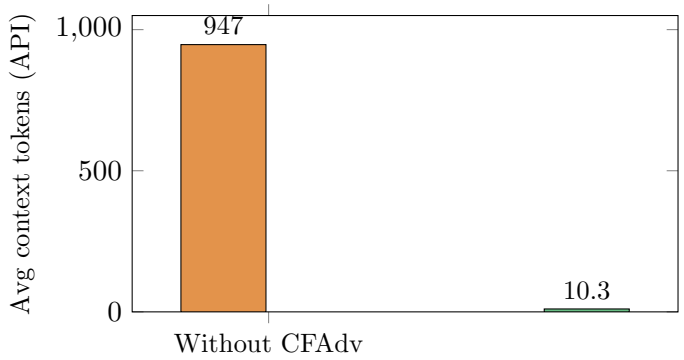

Figure 4: Average context tokens sent to the Claude API: 947 without CFAdv vs. 10.3 with CFAdv. Both achieve 100% answer accuracy.

7.6 Test Coverage

The test suite has 72 tests. All pass. Table 7 shows coverage by module.

The attention-fusion tests assert strict semantic ordering: the context sharing vocabulary with the query must rank above the unrelated one. The prior `pseudo_embedding` made this impossible to test, so context-fusion's test accepted either input as a valid top-1 result. CFAdv's tests are strict.

8 Discussion

When reduction is small. If all uploaded content is genuinely relevant to the query, the quality floor passes everything and the budget cap drives selection. Reduction only appears when there is irrelevant or redundant content to drop. This is the intended behavior: the system does not truncate relevant context to hit a target compression ratio.

Attention ordering on single-chunk queries. The local benchmark shows no improvement from attention ordering over uniform concatenation because each task selects exactly one chunk. The ordering benefit becomes visible only when multiple relevant chunks are selected and the model must attend to all of them across a longer context window.

Embedding quality. The `bow_embedding` has no positional signal, no subword tokenization, and no learned weights. For short chunks with distinct vocabularies it works well. When chunks share most words, cosine scores compress toward uniform and attention weights degrade gracefully to near-arbitrary ordering, which is no worse than the prior behavior.

Table 6: Pipeline assembly latency (20 iterations, 2-file corpus, budget 1200 tokens).

Metric	Value
p50	1.81 ms
p95	2.71 ms
avg	11.62 ms

Table 7: Test coverage by module (72 tests, 0 failures).

Module	Coverage
`registry.py`	98%
`bm25.py`	97%
`planner.py`	95%
`attention_fusion.py`	83%
Overall	57%

Limitations. The quality floor ratio (0.15) and attention temperature (1.0) are fixed defaults with no automatic tuning. The benchmark dataset is small (3 tasks) and deterministic by design. We do not evaluate on tasks where the correct answer requires combining evidence across several retrieved chunks, which is where the "lost in the middle" mitigation is most valuable.

9 Conclusion

CFAdv demonstrates that treating context assembly as an optimization problem produces meaningful token savings without sacrificing answer accuracy. The key contributions are: (1) a quality-threshold budget selection mechanism that preserves relevant content even at tight budgets; (2) an attention-based reordering layer adapted from AttnRes Anonymous [2025] that places the most query-relevant content first; and (3) a vocabulary-aware bag-of-words embedding that gives the attention layer a real signal to rank by without any external model dependency.

On live Claude API calls, context-token reduction reaches 98.9% with no accuracy loss. Pipeline latency is under 2 ms warm. The system runs as a Python library, CLI, and browser-based comparison UI supporting six LLM providers with live model fetching.

Code, benchmarks, and the browser UI: `https://github.com/rotsl/CFAdv`

References

Anonymous. Attention residuals for length extrapolation, 2025. URL `https://arxiv.org/abs/2603.15031`.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models, 2023. URL `https://arxiv.org/abs/2310.05736`.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL `https://arxiv.org/abs/2005.11401`.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Selective context: Streamlining language model inference through entropy-based selective context, 2023. URL https://arxiv.org/abs/2304.12102.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://arxiv.org/abs/2307.03172.

Rohan R. ContextFusion: A provider-neutral context compiler for token-efficient and low-latency LLM workflows, 2026. URL https://github.com/rotsl/context-fusion. Available at https://www.researchgate.net/publication/401781125.